Controlling and Automating a Model Railway with MersAG_Universal, OpenLCB and JMRI

***Important Note** :*
If you're reading this document directly from the Github repository you may find formatting errors. Please open the PDF in your MersAG_Universal PDF Documentation Folder.

This User Manual is presented in 4 parts each of which is available as a separate PDF download.

Part 1 gives a summary of the technologies we use and why, as model railway enthusiasts, we've chosen to use them. More importantly, it sets out why we believe that it gives the railway modelling community a fairly simple, low cost path to achieving high levels of reliable automation covering "realistic" turnout control and frog switching, signalling, sensing, block detection, lighting, animations and more.

Part 2 is aimed at modellers with little knowledge of electronics and/or, maybe, the fear of soldering!! We walk you through making a very simple, cheap "breadboard" to simulate a red/green signal from a simple push button switch. We then learn how easy it is, with our MersAG_Universal program, to integrate this with JMRI. Finally, and with just 2 more components and very little effort, we add slow-motion servo motor control. Our aim is to let you decide, with very little time and cost, if our way looks right for you.

In **Part 3** we'll add a few more components to build a "node" that can control multiple turnouts, signals etc and detect multiple sensors to control and automate all or part of your layout. This node will then be used to upgrade a region on an existing layout.

Finally, in **Part 4**, we'll add just 1 more component to enable several nodes to communicate over a CAN bus network that will enable you to control even very large complex layouts reliably.

Part 1 : Why you might want to start/continue your journey with us

1.1 Introduction 1

1.2 Controlling and Automating Trains, Track and Peripherals 1

1.3 About Us 1

1.4 MersAG-Universal 2

1.5 Integrating MersAG_Universal with other technologies 3

- 1.5.1 ESP32 microcontroller 3
- 1.5.2 LCC 3
- 1.5.3 OpenLCB 3
- 1.5.4 JMRI 3
- 1.5.5 I2C 4
- 1.5.6 CAN bus 4

Reference 4

1.1 Introduction

Model railroading started with the simple handcrafted wooden and metal toys of the early 1800s. It evolved throughout the 20th century with the innovations of electric, scaled models. The late 20th - early 21st century brought Digital Command Control (DCC) systems, enabling precise control of multiple trains on a single track while computer software allows for layout design and operational simulation. More recently, 3D printing has opened new possibilities for custom models and just about every accessory you can imagine.

Model Railroading is now a Global sophisticated hobby for collectors and enthusiasts who create elaborate layouts replicating real-world railroads, operations and landscapes. We're offered a wealth of commercial products to achieve just about anything we want to do with our locos and layout.

If we're just want to run our locos and control our layout maybe paying the price for the best of these products is the best route for us to take. But if we want to build our layouts to incorporate the control and automation that's now possible at surprisingly low cost we'll need to develop a practical understanding of the current technologies available to help achieve our goals.

Fortunately, many technologists and model railway hobbyists have collaborated to create open-source (i.e. free) products such as OpenLCB and JMRI that we can all benefit from. Our own group has explored many of these and used some in our own layouts. We've also developed an open-source computer program, MersAG_Universal, to integrate these and other low-cost products to control, automate and animate our model railways.

This User Guide shows you how to assemble the components you need, connect them and configure them to create a low-budget, sophisticated, scaleable, realistic model railway layout as simply as possible.

1.2 Controlling and Automating Trains, Track and Peripherals

In model railways, train control and track control refer to two distinct aspects of operation. While train control determines how each locomotive behaves, track control governs the routes they can take. With more advanced control systems the track can additionally control the conditions under which the locomotives can move, ensuring smooth, coordinated and realistic operation.

Traditionally, train control has been by supplying the track with a DC current that controls the train's speed and direction. Modern train control can be achieved through several systems, with Digital Command Control (DCC) being the most popular. DCC control units send both power and digital signals through the rails, allowing independent control of multiple locomotives, some fitted with sound decoders, lights and other features.

Track control is increasingly using low-cost microcontrollers such as Arduinos and ESP32s with programs, called sketches, to control turnouts/points, signals and animations. They can also read spot and block detectors and other sensors to enable even greater levels of automation that enable trains to run along pre-programmed routes, stop at stations and respond to track conditions safely and without manual intervention. These systems can also interact with DCC train controllers to modify the speed and direction of specific trains depending on track conditions, or to operate trains through schedules.

For very large and/or complex layouts multiple microcontrollers can be networked together to enable very large, complex model railway layouts to operate safely and with a very high degree of automation and realism. As you might imagine, this is our goal!

1.3 About Us

The OpenLCB (Open-source Layout Control Bus) Universal project started as a collaboration of three MERG (Model Electronic Railway Group) members David Harris (Lead software developer), David Ellis (Software proposer), and John Holmes (Group Leader and software developer). During the development the software was being showcased at the Merseyside Area Group (MersAG), a sub group of MERG. As people started to see the potential Pete Colley offered to take on a role of making the documentation beginner friendly. This needed the software to be locked down to assist with this task. To allow this the group decided to use the name MersAG_Universal.

The MersAG group meets two times a month, once in a face to face meeting, and once by Zoom. The group has a wide range of interests in railway modelling.

1.4 MersAG-Universal

MersAG_Universal is an open-source sketch that enables DC or DCC model railway layouts to be controlled and automated through **JMRI** using **OpenLCB** as the messaging system. Control is effected via the following specified hardware components that are inexpensive, widely available and easy to connect on one or a network of nodes each comprising :

- an ESP32-DevKitC-1 with 30 pin expansion board;
- 0, 1, 2 or 3 PCA9685 16-channel, 12-bit PWM expansion board enabling up to 16, 32 or 48servos to control turnouts/points, semaphores, animations and more;
- Up to 4 MCP23017 16-bit I/O expansion boards enabling, with 8 available I/O pins on the 30 pin expansion board for things like frog switching, signal and lighting LEDs, block, spot and other detectors, and more

together with other components as required by the user.

Key features of MersAG_Universal include :
- Easy to configure using JMRI;
- Compatible with Windows, MacOS and Linux;
- Enables individual slow servo control offering more realistic movement of turnouts/points, animations etc;
- Standalone operation for control without being connected to a computer once configured;
- Full or partial automation through JMRI;
- Budget-level advanced control and automation.

Users can configure a MersAG_Universal node to control :
- 1 x PCA9685 and up to 4 x MCP23017 : 16 PWM units and up to 82 I/O pins; or
- 2 x PCA9685 and up to 3 x MCP23017 : 32 PWM units and up to 56 I/O pins

MersAG_Universal Nodes offer maximum flexibility and versatility :
- A node can have just 1 PCA9685 to control up to 16 servos, animations etc and 8 In/Outputs (connected directly to the ESP32 Expansion Board)
- A single MCP23017 can be added to give 24 In/Outputs to control frog relay switches, LEDs, etc and read spot and block detector
- Another 3 MCP23017s can be added to give a total of 72 In/Outputs enabling lots of signals, sensors, lights to be controlled/read
- A 2nd PCA9685 can be added instead of an MCP23017 to control up to 32 servos, animations etc with up to 56 In/Outputs
- Multiple Nodes can be connected together in a reliable CAN bus network of tens of PCA9685s and hundreds of MCP23017s to control large very complex layouts.

MersAG_Universal supports three modes of operation :

1 **Standalone**. In this mode, MersAG_Universal is configured to the User's requirements using JMRI. Once configured JMRI and the computer can be disconnected and the layout can be controlled, monitored and partly automated using push-buttons, switches, LEDs etc on mimic panels.

2 **Automated control**. In this mode, MersAG_Universal is again configured to the User's requirements using JMRI. JMRI is then used to continually monitor, control and automate the layout to enable multiple locomotives to run safely around user-defined routes and routines.

3 **Network control**. In this mode, a number of "nodes" are connected together by a CAN-based bus network that considerably reduces the wiring and enables large, complex layouts to be fully controlled and automated without the problems associated with electrical interference, noise etc.

The protocol also offers the opportunity to enable other communication methods such as WiFi in the future.

1.5 Integrating MersAG_Universal with other technologies

From the day we started our journey we've sought to find well tried and tested open-source technologies and bring them together to create a reliable, robust, scaleable solution to the challenge of controlling and automating model railways. Following is a brief overview of the key technologies we use.

1.5.1 ESP32 microcontroller

We initially used Arduinos as our microcontroller but soon realised that they didn't have the power and capacity we needed. We tried a few alternative solutions and settled on the ESP32 Devkit 1 USB Type C Development Board as it offered the power, performance and reliability that we were looking for. It's also compatible with the Arduino IDE allowing us to implement the sketches we'd been developing. The 30Pin DEVKIT V1 Power Supply Board gave us integrated voltage regulation to increase reliability and easy-to-access pinouts.

The ESP32 chip has dual-core processing and built-in Wi-Fi and Bluetooth capabilities, enabling wireless communication and making it ideal for automation and monitoring. It also has built in I^2C networking making it easy to connect sensors, actuators and displays without complex wiring.

1.5.2 LCC

LCC in model railway refers to **Layout Command Control**, a new standard for layout control developed by the **National Model Railroad Association** (NMRA) and the **OpenLCB group**. It is an open-source protocol that allows for the integration of various model railway control systems, such as DC, DCC, radio, live steam and others. LCC is designed to be scalable and powerful enough to control large layouts, including those in museums or built by modular clubs. It operates in parallel to the train control system and can be thought of as doing for layout control what DCC has done for train control. LCC is compatible with many existing model railway control systems and and allows for wireless locomotive control over the LCC bus through the use of JMRI for MersAG_Universal.

1.5.3 OpenLCB

In model railroading, **LCB** stands for **Layout Control Bus**. It's a messaging system that allows model railroad products to communicate with each other, enabling control of various components such as lights, signals and turnouts. The OpenLCB Group has developed this open-source connection standard, which is designed to work alongside DCC (Digital Command Control) systems.

LCB is particularly useful for controlling large layouts, such as those found in museums or built by modular clubs, as it simplifies wiring and enhances control precision. The system connects units called 'nodes', which are typically circuit boards that take inputs and control outputs. These nodes send messages to each other using a few wires run between them, known as a 'bus'. The OpenLCB Group has also developed LCC (Layout Command Control), which is an open standard for a Layout Control Bus agreed by the NMRA. For MersAG_Universal, openLCB is the same as LCC but it allows us to not use RJ45 connectors as we hard wire nodes onto the CAN bus.

1.5.4 JMRI

JMRI, or **Java Model Railroad Interface**, is an open-source software suite designed for model railroaders. It allows users to control LED lights, horns, and switch the railway of hobbyist's open-source or closed-sourced trains. JMRI is a suite of tools distributed via a single download, with the most popular tools being **DecoderPro** for programming Digital Command Control (DCC) decoders and **PanelPro** for controlling layouts. JMRI is compatible with virtually all DCC systems and provides powerful tools for operations, making it a valuable resource for model railway enthusiasts in the UK and beyond.

We use JMRI to configure MersAG_Universal to individual requirements. We also integrate it when using MersAG_Universal in Automated and Network Control modes.

1.5.5 I²C

I²C (Inter-Integrated Circuit), pronounced “I squared C” is a communication protocol used in model railways to connect various components such as turnouts, signals, sensors, lights and sounds. It allows for the control of these devices through the EX-CommandStation, which is the central control unit for DCC-EX model railroading. I²C devices communicate using two signal lines: SCL (clock) and SDA (data), which are connected to a common ground. Each I²C device must have a unique address, typically in hexadecimal format, to be detected and configured by the software.

I²C does have a limited range so we don’t recommended it on buses of more than 1m in length, although we acknowledge that some users successfully use I²C buses of more than 1m. We use it for communication between devices within a ‘node' using very short leads of less than a 100mm'.

1.5.6 CAN bus

CAN bus was developed by Robert Bosch GmbH in the 1980s to reduce wiring complexity in automobiles and enable reliable, safe communication between multiple ECUs, such as engine control, brakes, airbags, and infotainment systems and is now widely used in automotive, industrial, robotics and other real-time applications

CAN bus is inherently resistant to interference by electrical noise and is scaleable, allowing new nodes to be added without rewiring the whole system. It’s also bi-directional enabling modules to both send and receive events.

In model railways, CAN bus is a simple two-wire digital communication system (CAN-H and CAN-L) used to link controllers, decoders, sensors, and accessories on a layout, reducing wiring and enabling reliable, fast, bidirectional data exchange over long distances. We use it to connect nodes together in complex and/or large expandable layouts.

Reference

Node

In our context we use the word node to mean an ESP32 and expansion board connected to PCA9685(s) and MCP23017(s) expansion boards via an I²C bus. Other components such as buck converters to supply 3.3v from a 5 or 12v ring, or relay boards to connect to turnout frogs can form part of the node as well (see our User Guide “Controlling and Automating your Model Railway with MersAG_Universal, OpenLCB and JMRI : Part 3”) . A network is created by adding a CAN bus module which interfaces with a transceiver on each node. Each node on the network must have a unique address (see our User Guide “Controlling and Automating a Model Railway with MersAG_Universal, OpenLCB and JMRI : Part 4”)